

Article

Reusing Policy-as-Code Across CI/CD and Kubernetes Admission Control: An Empirical Assessment of Governance Consistency

Luís Nogueira *  and Alice Resende 

School of Engineering (ISEP), Polytechnic Institute of Porto (P.PORTO), 4249-015 Porto, Portugal;
1211518@isep.ipp.pt

* Correspondence: lmn@isep.ipp.pt; Tel.: +351-228340500

Abstract

Cloud-native software-delivery pipelines increasingly rely on Policy-as-Code (PaC) to automate security, compliance, and governance enforcement. Although Policy-as-Code is widely adopted within Continuous Integration (CI) pipelines and Kubernetes admission-control frameworks, governance requirements are often implemented independently, potentially increasing maintenance effort and creating opportunities for policy drift. Despite the growing adoption of Policy-as-Code, comparatively little empirical evidence exists regarding the reuse of a shared policy-definition layer across complementary enforcement stages within the software-delivery lifecycle. This paper presents and empirically evaluates a reusable multi-stage Policy-as-Code enforcement model based on a shared policy-definition layer implemented using the Open Policy Agent (OPA) framework and its Rego policy language. Rather than proposing a new Policy-as-Code technology, the study investigates whether a shared policy-definition layer can support consistent policy enforcement across Continuous Integration validation and Kubernetes admission control. The model was evaluated using Conftest and OPA Gatekeeper through a structured experimental study comprising 29 Kubernetes manifests, 37 experimental scenarios, eight Kubernetes resource types, and 261 policy assertions covering representative cloud-native workload-governance requirements. Within the evaluated dataset, all intentionally introduced insecure configurations were correctly identified without observed false positives or false negatives. The shared policy-definition layer was successfully reused across both validation stages, while Kubernetes admission control mitigated all evaluated CI bypass scenarios by providing an independent deployment-time enforcement boundary. The results demonstrate that a shared policy-definition layer can support consistent policy enforcement across complementary enforcement stages while enabling policy reuse without requiring duplicate policy implementations within the evaluated environment. More broadly, the study contributes empirical evidence supporting policy reuse as a governance strategy for cloud-native software delivery and provides a reproducible foundation for future investigations involving larger datasets, broader governance-policy portfolios, alternative Policy-as-Code ecosystems, and production-scale deployments.



Academic Editor: Paolo Bellavista

Received: 18 June 2026

Revised: 10 July 2026

Accepted: 16 July 2026

Published: 17 July 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article

distributed under the terms and

conditions of the [Creative Commons](#)

[Attribution \(CC BY\)](#) license.

Keywords: policy-as-code; governance consistency; policy reuse; continuous integration; Kubernetes admission control; DevSecOps; governance-as-code; cloud-native security; open policy agent; rego

1. Introduction

The widespread adoption of cloud-native architectures, Infrastructure-as-Code (IaC), and Kubernetes-based deployment platforms has transformed modern software-delivery processes. Organizations increasingly rely on Continuous Integration and Continuous Delivery (CI/CD) pipelines to accelerate software releases while maintaining security, compliance, and operational governance requirements [1–4]. As software-delivery pipelines become more automated and distributed, ensuring consistent governance enforcement across multiple stages of the software-delivery lifecycle has emerged as a significant challenge.

DevSecOps extends traditional DevOps practices by integrating security and governance activities directly into software development and deployment workflows [2]. Rather than relying on isolated security reviews performed late in the lifecycle, DevSecOps promotes continuous verification, automated compliance checking, and policy-driven governance throughout software delivery. However, several studies report that organizations continue to face difficulties arising from fragmented governance controls, duplicated policy implementations, inconsistent enforcement mechanisms, and the operational complexity of managing security and compliance requirements across heterogeneous toolchains [5–8].

Policy-as-Code (PaC) has emerged as a promising approach for expressing governance requirements as executable policies that can be versioned, tested, reviewed, and automatically enforced throughout the software-delivery lifecycle [9–11]. Its adoption has expanded beyond security assurance to encompass compliance automation, infrastructure governance, and identity-management enforcement, reflecting its growing role in cloud-native software delivery [12,13]. Among the available Policy-as-Code technologies, Open Policy Agent (OPA) [14] and its declarative policy language Rego have become widely adopted due to their portability and ability to operate across diverse enforcement environments [10,11].

In Kubernetes environments, Policy-as-Code is commonly enforced through two complementary mechanisms. The first performs pre-deployment validation during Continuous Integration (CI), where deployment artifacts are evaluated before progressing through the delivery pipeline. The second performs deployment-time validation through Kubernetes admission control, where workload definitions are inspected immediately before acceptance by the cluster. Previous studies have demonstrated the effectiveness of both approaches for improving governance assurance and reducing configuration-related risks in cloud-native environments [15–17]. However, existing work primarily evaluates these enforcement mechanisms independently and provides limited empirical evidence regarding the reuse of a shared policy-definition layer across complementary enforcement stages.

This gap is particularly relevant because organizations frequently maintain separate policy implementations targeting equivalent governance requirements across CI pipelines, deployment platforms, and runtime environments. Such duplication can increase maintenance effort, complicate policy evolution, and create opportunities for policy drift, potentially leading to inconsistent policy enforcement across different stages of the software-delivery lifecycle.

To address this gap, this paper presents and empirically evaluates a reusable multi-stage Policy-as-Code enforcement model based on a shared policy-definition layer implemented using Rego policies executed during both CI validation and Kubernetes admission control. Rather than proposing a new Policy-as-Code technology, the study investigates whether maintaining a shared policy-definition layer can support consistent policy enforcement across complementary enforcement stages.

The proposed model was implemented using Conftest and OPA Gatekeeper [18] and evaluated through a structured experimental study comprising 29 Kubernetes manifests, 37 experimental scenarios, eight Kubernetes resource types, and 261 policy assertions.

The evaluation investigates the practical feasibility of policy reuse across complementary enforcement stages, the characteristics of layered policy enforcement, and the ability of Kubernetes admission control to mitigate CI bypass scenarios.

The main contributions of this work are:

- A reusable multi-stage Policy-as-Code enforcement model centered on a shared policy-definition layer reused across Continuous Integration validation and Kubernetes admission control.
- A structured empirical evaluation comprising 29 Kubernetes manifests, 37 experimental scenarios, eight Kubernetes resource types, and 261 policy assertions designed to systematically assess policy reuse across complementary enforcement stages.
- Empirical evidence that a shared policy-definition layer can support consistent policy enforcement across Continuous Integration validation and Kubernetes admission control while providing an additional deployment-time enforcement boundary against the evaluated CI bypass scenarios.

All policies, manifests, experimental scenarios, workflows, and evaluation results are publicly available through an open replication package.

The remainder of this paper is organized as follows. Section 2 reviews the literature on DevSecOps governance, Policy-as-Code, and Kubernetes policy enforcement. Section 3 presents the research methodology and experimental design. Section 4 introduces the proposed reusable multi-stage Policy-as-Code enforcement model. Section 5 describes the experimental evaluation and presents the results. Section 6 discusses the implications, limitations, and broader significance of the findings. Finally, Section 7 concludes the paper and outlines future research directions.

2. Background and Related Work

2.1. DevSecOps Governance and Security Automation

The widespread adoption of DevOps practices has transformed software delivery by promoting automation, collaboration, and continuous integration throughout the software development lifecycle [1,2]. As organizations increasingly adopt cloud-native architectures and Infrastructure-as-Code (IaC), software-delivery pipelines have become more complex, distributed, and highly automated, creating new challenges for maintaining security, compliance, and governance throughout the software-delivery lifecycle [3,4].

To address these challenges, DevSecOps extends DevOps principles by integrating security and governance activities directly into development and deployment workflows [2,19]. Rather than treating security as a separate activity performed late in the lifecycle, DevSecOps promotes continuous verification, automated compliance checking, and policy-driven governance throughout the delivery process. Consequently, governance decisions are increasingly expected to be performed automatically and consistently across multiple stages of the software-delivery lifecycle.

Despite its growing adoption, ensuring consistent policy enforcement across heterogeneous software-delivery pipelines remains a significant challenge. Security controls are frequently implemented through multiple tools operating at different stages of the delivery pipeline, resulting in duplicated policy implementations, fragmented enforcement mechanisms, and potentially increased maintenance effort [5–8]. These challenges become particularly relevant in cloud-native environments, where deployment decisions are increasingly driven by declarative configuration artifacts evaluated by heterogeneous validation and enforcement mechanisms.

Collectively, these studies demonstrate that automating governance throughout the software-delivery lifecycle is essential for achieving scalable security and compliance in

cloud-native environments. However, existing research has primarily focused on improving governance within individual delivery stages, providing comparatively little guidance on how governance policies can be maintained consistently across complementary enforcement stages. This limitation motivates further research into Policy-as-Code approaches that support the consistent reuse of governance requirements throughout the software delivery lifecycle.

2.2. Policy-as-Code and Governance Automation

Policy-as-Code (PaC) has emerged as a promising approach for addressing governance challenges in modern software-delivery environments. The fundamental idea is to express governance requirements as executable policies that can be versioned, tested, reviewed, and automatically enforced using engineering practices similar to those applied to software artifacts [9–11].

By transforming governance requirements into machine-readable policies, organizations can improve consistency, traceability, and automation throughout software-delivery processes. Recent work highlights the growing adoption of Policy-as-Code for compliance management, access control, infrastructure governance, and continuous assurance [12,13]. These characteristics have established Policy-as-Code as one of the key enabling technologies supporting governance automation in cloud-native environments.

Several complementary research directions have also emerged. Haverinen et al. [9] investigate Security-as-Code models for representing governance requirements as executable artifacts that can be integrated into automated software-delivery processes. More broadly, existing research highlights the growing role of Policy-as-Code and Security-as-Code approaches in improving compliance automation, traceability, and continuous assurance across cloud-native software-delivery environments.

Among the available technologies, Open Policy Agent (OPA) and its declarative policy language Rego have become widely adopted due to their flexibility, portability, and ability to operate across heterogeneous execution environments [10,11,14]. These characteristics make OPA particularly suitable for enforcing governance requirements throughout cloud-native software-delivery pipelines. Moreover, its independence from the underlying enforcement mechanism enables the same policy definitions to be evaluated across different validation stages, making OPA an appropriate platform for investigating policy reuse through a shared policy-definition layer.

Although existing studies clearly demonstrate the benefits of Policy-as-Code for governance automation, they predominantly focus on policy definition, policy languages, enforcement frameworks, or individual execution contexts. Comparatively little empirical attention has been devoted to evaluating whether a shared policy-definition layer can support consistent policy enforcement across complementary enforcement stages. Consequently, the research challenge addressed in this work no longer concerns the availability of Policy-as-Code technologies themselves, but rather understanding whether reusable policy definitions can be consistently enforced throughout the software-delivery lifecycle.

2.3. Policy Enforcement in Kubernetes Environments

Kubernetes has become one of the dominant platforms for orchestrating cloud-native applications, leading to growing interest in automated governance and policy-enforcement mechanisms. The extensive configuration surface exposed by Kubernetes manifests increases the likelihood of security misconfigurations and policy violations, motivating the adoption of automated validation techniques [15,16].

Existing approaches include static configuration analysis, vulnerability scanning, runtime monitoring, and admission-control validation. Among these mechanisms, admission

control plays a particularly important role because it enables policy evaluation immediately before resources are accepted by the cluster. Admission controllers intercept requests after authentication and authorization but before resource persistence, providing a deployment-time enforcement boundary capable of rejecting non-compliant workloads [20]. Through validating admission webhooks, governance policies can be enforced independently of upstream development and deployment activities.

Previous studies demonstrate the effectiveness of policy-driven admission-control mechanisms for improving governance assurance and reducing configuration-related risks in Kubernetes environments [17]. Collectively, these studies establish admission control as an effective deployment-time governance mechanism capable of preventing non-compliant workloads from reaching production clusters while complementing upstream software-delivery activities.

However, existing research has predominantly evaluated admission control as an isolated deployment-time enforcement mechanism. Comparatively little empirical attention has been devoted to understanding its relationship with earlier validation stages, such as Continuous Integration, or to investigating whether a shared policy-definition layer can support consistent policy enforcement across both execution contexts. Consequently, although Kubernetes admission control is now well established as a deployment-time governance mechanism, its role within broader multi-stage Policy-as-Code governance strategies remains relatively underexplored.

2.4. Policy Enforcement Across the Software-Delivery Lifecycle

Modern software-delivery pipelines provide multiple opportunities for policy enforcement. Governance policies may be evaluated during source-code validation, Infrastructure-as-Code (IaC) analysis, Continuous Integration (CI), deployment verification, Kubernetes admission control, or runtime operation. These enforcement points differ in their objectives, execution context, and timing, collectively contributing to a defense-in-depth strategy for cloud-native software delivery by establishing complementary governance-validation boundaries throughout the software-delivery lifecycle.

Several studies highlight the benefits of performing policy validation during Continuous Integration, where governance violations can be detected before deployment artifacts progress through the delivery process [6,9]. Early validation shortens feedback cycles, enables developers to correct configuration issues before deployment, and reduces the propagation of non-compliant artifacts through subsequent stages of the delivery pipeline.

Similarly, Kubernetes admission-control mechanisms provide an independent deployment-time enforcement boundary capable of preventing non-compliant workloads from reaching production environments [17]. Unlike CI validation, admission control evaluates deployment requests immediately before resource persistence within the cluster, allowing governance policies to be enforced independently of upstream development and deployment activities.

Although Continuous Integration validation and Kubernetes admission control pursue the same governance objectives, they operate at different stages of the software-delivery lifecycle and therefore provide complementary rather than competing enforcement capabilities. Continuous Integration primarily supports early developer feedback and defect prevention, whereas Kubernetes admission control performs authoritative deployment-time validation immediately before workload admission. Together, these complementary enforcement stages strengthen governance assurance by establishing multiple validation boundaries throughout the software-delivery lifecycle.

Despite this complementarity, existing research has predominantly investigated these enforcement mechanisms in isolation. The literature provides extensive discussion of

Policy-as-Code technologies, governance automation, and Kubernetes admission control, yet comparatively little empirical evidence is available regarding whether reusable policy definitions can be consistently enforced across complementary stages of the software-delivery lifecycle. Consequently, the interaction between multiple policy-enforcement stages remains relatively underexplored, particularly with respect to policy reuse, layered governance assurance, and resilience against CI bypass scenarios. These observations directly motivate the research gap discussed in the following subsection.

2.5. Research Gap

The reviewed literature demonstrates significant progress in DevSecOps governance, Policy-as-Code, compliance automation, and Kubernetes policy enforcement. Existing studies have established the value of executable policies, automated governance, and deployment-time validation for improving security and compliance throughout the software-delivery lifecycle. However, these contributions have largely been evaluated within individual enforcement contexts, with most research focusing either on Continuous Integration (CI) validation or Kubernetes admission control.

Previous work has shown that Policy-as-Code can support governance automation and compliance assurance, while Kubernetes admission-control mechanisms provide an effective means of preventing non-compliant workloads from reaching production environments. Despite these advances, comparatively little empirical research has examined whether reusable policy definitions can be consistently enforced across complementary enforcement stages within the same software-delivery lifecycle.

This gap is particularly relevant in cloud-native environments, where governance requirements are frequently enforced through heterogeneous toolchains operating at different stages of the delivery process. In practice, organizations often maintain separate policy implementations targeting equivalent governance requirements, which can increase maintenance effort, complicate policy evolution, and create opportunities for policy drift.

The research gap addressed in this work, therefore, does not concern the availability of Policy-as-Code technologies or enforcement mechanisms themselves. Mature solutions already exist for policy definition, Continuous Integration validation, and Kubernetes admission control. Instead, the gap lies in the limited empirical evidence regarding whether reusable policy definitions can be consistently enforced across complementary enforcement stages while preserving equivalent governance behavior throughout the software delivery lifecycle.

Furthermore, although the potential organizational benefits of policy reuse are frequently discussed in the literature, their practical feasibility has received comparatively little empirical attention. In particular, it remains unclear whether a shared policy-definition layer can preserve consistent policy enforcement across multiple validation stages and whether deployment-time admission control effectively complements earlier Continuous Integration validation by mitigating governance failures caused by CI bypass scenarios.

To address this gap, the present study evaluates a reusable multi-stage Policy-as-Code enforcement model centered on a shared policy-definition layer implemented using Rego policies and executed across complementary enforcement stages. Rather than proposing a new Policy-as-Code technology, the study empirically investigates whether policy reuse can support consistent policy enforcement throughout the software-delivery lifecycle. The following sections introduce the proposed architectural model and present a structured experimental evaluation designed to assess the practical feasibility of layered Policy-as-Code enforcement in cloud-native software-delivery pipelines.

3. Research Methodology

3.1. Research Method

This study follows a Design Science Research (DSR) approach, which is well suited to investigations involving the design, implementation, and empirical evaluation of artifacts intended to address practical engineering problems while generating scientifically grounded knowledge [21,22]. In this work, the artifact consists of a reusable multi-stage Policy-as-Code enforcement model that enables the execution of a shared policy-definition layer across complementary enforcement stages of a cloud-native software-delivery pipeline.

The research process comprised three sequential phases. First, a structured literature review was conducted to analyze the state of the art in DevSecOps governance, Policy-as-Code, and Kubernetes policy enforcement. This phase established the scientific context of the study, identified limitations in existing approaches, and motivated the research gap presented in Section 2. Second, the proposed Policy-as-Code enforcement model was designed and implemented using Open Policy Agent (OPA), Rego, Conftest, and OPA Gatekeeper. Finally, the resulting artifact was evaluated through a structured experimental study based on representative Kubernetes governance scenarios.

The experimental evaluation does not aim to compare alternative Policy-as-Code technologies, policy languages, or enforcement engines. Rather, it investigates whether maintaining a shared policy-definition layer across complementary enforcement stages can support consistent policy enforcement throughout the software-delivery lifecycle. Consequently, the same Rego policy definitions were reused throughout all experiments, allowing the evaluation to isolate the effects of policy reuse from those of the underlying enforcement technologies.

The experimental methodology was explicitly designed to ensure traceability between the research questions, the experimental scenarios, the collected evidence, and the evaluation metrics. Each research question is addressed through dedicated experimental scenarios and objective evaluation criteria, enabling the systematic assessment of policy-detection capability (RQ1), policy coverage across complementary enforcement stages (RQ2), and the effectiveness of deployment-time admission control in mitigating governance failures caused by CI bypass scenarios (RQ3).

By combining artifact design with a structured empirical evaluation, the adopted Design Science Research methodology not only demonstrates the practical feasibility of the proposed enforcement model but also provides reproducible empirical evidence regarding policy reuse across complementary enforcement stages.

Figure 1 summarizes the overall research process adopted in this study.

3.2. Experimental Environment

The experimental environment was designed to reproduce a representative cloud-native software-delivery pipeline in which identical Policy-as-Code definitions are enforced during both Continuous Integration (CI) validation and Kubernetes admission control. The environment combines widely adopted open-source technologies representative of contemporary DevSecOps practice while allowing the experimental evaluation to focus on policy reuse rather than on differences between individual enforcement technologies.

The Policy-as-Code implementation is based on the Open Policy Agent (OPA) framework and its declarative policy language Rego. A single set of Rego policy definitions constitutes the shared policy-definition layer reused throughout the evaluation. During Continuous Integration, policy evaluation is performed using Conftest integrated into GitHub (3.18.3) Actions workflows, enabling Kubernetes manifests to be validated before deployment artifacts progress through the delivery pipeline. During deployment, the same

policy definitions are evaluated through OPA Gatekeeper using Kubernetes validating admission webhooks.

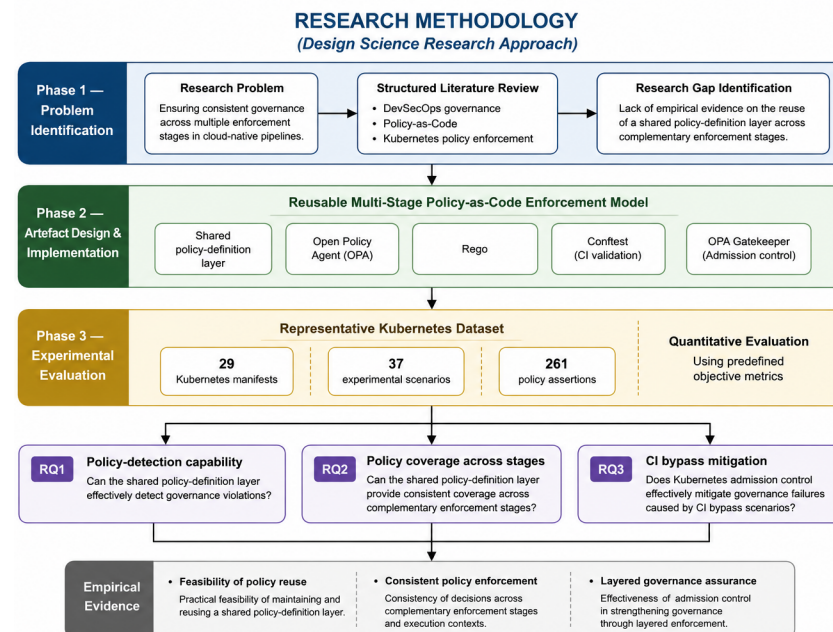


Figure 1. Overview of the Design Science Research methodology adopted in this study. The research process comprises three sequential phases: (i) problem identification through a structured literature review leading to the identification of the research gap; (ii) design and implementation of the reusable multi-stage Policy-as-Code enforcement model centered on a shared policy-definition layer; and (iii) structured experimental evaluation using a representative Kubernetes dataset. The evaluation addresses the three research questions concerning policy-detection capability (RQ1), policy coverage across complementary enforcement stages (RQ2), and the effectiveness of Kubernetes admission control in mitigating governance failures caused by CI bypass scenarios (RQ3). Together, these activities provide reproducible empirical evidence regarding the practical feasibility of policy reuse across complementary enforcement stages.

The evaluation platform consists of a GitHub Actions Continuous Integration pipeline executing Conftest and a Kubernetes cluster configured with OPA Gatekeeper. Both enforcement stages evaluate identical deployment artifacts without modification, ensuring that any differences in enforcement behavior arise from the execution context rather than from differences in policy implementation.

The environment was intentionally designed to isolate the effects of policy reuse from those of the underlying enforcement technologies. Consequently, the study evaluates the behavior of the shared policy-definition layer across complementary enforcement stages rather than comparing alternative policy engines, policy languages, or Kubernetes governance frameworks. This design strengthens the internal validity of the study by ensuring that the underlying policy implementation remains constant throughout the evaluation, allowing observed differences to be attributed solely to the execution context.

Table 1 summarizes the software components used throughout the study. These technologies were selected because they are widely adopted within contemporary cloud-native software-delivery pipelines and support the evaluation of a common policy-definition layer across complementary enforcement stages.

Table 1. Experimental environment.

Component	Technology
CI platform	GitHub Actions
CI policy evaluation	Conftest
Shared policy-definition layer	Rego policies
Policy engine	Open Policy Agent (OPA)
Admission controller	OPA Gatekeeper
Target platform	Kubernetes
Evaluated artifacts	Kubernetes manifests

3.3. Dataset Construction

The evaluation dataset was specifically constructed to support the systematic empirical investigation of policy reuse across complementary enforcement stages. Rather than aiming to statistically represent Kubernetes deployments observed in practice, the dataset was designed to provide controlled and reproducible experimental conditions in which the expected outcome of every policy evaluation is known *a priori*. This design strengthens the internal validity of the study by enabling precise verification of policy behavior while ensuring complete traceability between deployment artifacts, expected policy decisions, and observed outcomes.

The dataset was constructed using Kubernetes resource specifications, official Kubernetes documentation, and representative cloud-native deployment patterns as design references. Each manifest was intentionally created to exercise one or more governance requirements commonly encountered in Kubernetes environments. Collectively, the evaluated artifacts cover representative governance concerns, including workload configuration, namespace governance, container security, and deployment validation, corresponding to policy classes frequently enforced within contemporary DevSecOps pipelines.

To ensure systematic coverage of different governance conditions, the deployment artifacts were organized into five complementary categories. Each category was designed to address a specific evaluation objective and to provide evidence for one or more research questions, as summarized in Table 2.

Table 2. Experimental dataset categories and evaluation objectives.

Category	Evaluation Objective	RQ
Compliant workloads	Baseline policy validation and compliance verification	RQ1
Negative controls	False-positive assessment using unrelated resources	RQ1
Single-policy violations	Evaluation of individual policy-detection capability	RQ1
Multi-policy violations	Evaluation of policy coverage under layered enforcement	RQ2
CI bypass scenarios	Evaluation of admission-control behavior after CI bypass	RQ3

The complete dataset comprises 29 Kubernetes manifests spanning eight Kubernetes resource types and evaluated through 37 experimental scenarios, resulting in a total of 261 policy assertions. Every experimental scenario specifies its expected outcome before execution, allowing the observed behavior to be compared directly against predefined expectations. This design establishes complete traceability between deployment artifacts, policy assertions, research questions, and the corresponding experimental evidence.

The CI bypass scenarios deserve particular attention because they reproduce situations in which deployment artifacts reach the Kubernetes API server without undergoing upstream Continuous Integration validation. Although intentionally simplified, these scenarios represent a realistic deployment condition in which Kubernetes admission control acts as an independent deployment-time enforcement boundary capable of preventing non-compliant workloads from reaching the cluster.

Although the dataset was intentionally designed for controlled experimentation rather than statistical representativeness, its systematic construction provides a rigorous and fully reproducible basis for evaluating policy reuse across complementary enforcement stages. Consequently, the reported findings should be interpreted as empirical evidence regarding the practical feasibility of maintaining a shared policy-definition layer under well-defined experimental conditions rather than as a statistical characterization of governance violations in production Kubernetes environments.

3.4. Research Questions

The experimental evaluation is guided by three research questions derived directly from the research gap identified in Section 2. Collectively, these questions investigate the feasibility of reusing a shared policy-definition layer across complementary enforcement stages. Rather than comparing individual Policy-as-Code technologies, policy languages, or enforcement engines, the research questions examine three complementary aspects of the proposed approach: policy-detection capability, policy coverage across multiple enforcement stages, and the contribution of deployment-time admission control as an independent enforcement boundary.

- **RQ1: Can reusable Policy-as-Code definitions accurately detect insecure Kubernetes configurations automatically?**

This research question investigates the detection capability of the shared policy-definition layer. Specifically, it examines whether intentionally introduced governance violations are correctly identified while compliant workloads and unrelated Kubernetes resources remain free from false-positive detections. The objective is to establish the effectiveness of reusable Policy-as-Code definitions as an automated governance mechanism under controlled experimental conditions.

- **RQ2: Does multi-stage Policy-as-Code enforcement provide broader policy coverage than single-stage enforcement?**

This research question investigates the extent to which combining Continuous Integration validation with Kubernetes admission control enables governance requirements to be enforced across complementary stages of the software-delivery lifecycle. It focuses on the complementary role of each enforcement stage and on the practical feasibility of reusing a common policy-definition layer throughout the software-delivery lifecycle.

- **RQ3: Can Kubernetes admission control mitigate governance failures caused by CI bypass scenarios?**

This research question investigates the role of Kubernetes admission control as an independent deployment-time enforcement boundary. Specifically, it examines whether governance enforcement is preserved when Kubernetes manifests reach the Kubernetes API server without undergoing upstream Continuous Integration validation, thereby assessing the contribution of layered policy enforcement under CI bypass conditions.

Collectively, these research questions address the central objective of the study: to provide empirical evidence regarding the practical feasibility of reusing a shared policy-definition layer across complementary enforcement stages while preserving consistent governance enforcement throughout the cloud-native software-delivery lifecycle. The following subsections describe the experimental environment, dataset construction, evaluation metrics, and experimental design used to investigate each research question systematically.

3.5. Evaluation Metrics

The experimental evaluation is based on a set of quantitative metrics designed to provide objective and reproducible evidence for answering the research questions. Collectively, these metrics assess three complementary aspects of the proposed reusable Policy-as-Code enforcement model: policy-detection capability (RQ1), policy coverage across complementary enforcement stages (RQ2), and the effectiveness of Kubernetes admission control in mitigating CI bypass scenarios (RQ3). Table 3 summarizes the evaluation metrics adopted throughout the study.

Table 3. Evaluation metrics.

Metric	Definition	Unit
Detection Rate (DR)	Percentage of expected policy violations correctly detected by the evaluated enforcement stage.	%
False Positive Rate (FPR)	Percentage of compliant resources incorrectly reported as policy violations.	%
False Negative Rate (FNR)	Percentage of expected policy violations that remain undetected.	%
Policy Coverage (PC)	Percentage of evaluated governance policies enforced by each enforcement stage.	%
CI Bypass Protection (CBP)	Percentage of CI bypass scenarios correctly rejected by Kubernetes admission control.	%

The Detection Rate (DR) measures the proportion of expected policy violations correctly identified by the evaluated enforcement stage:

$$DR = \frac{TP}{TP + FN} \times 100 \quad (1)$$

where TP denotes the number of correctly detected policy violations and FN the number of expected policy violations that remain undetected.

The False Positive Rate (FPR) measures the proportion of compliant resources incorrectly classified as policy violations:

$$FPR = \frac{FP}{FP + TN} \times 100 \quad (2)$$

where FP denotes the number of compliant resources incorrectly classified as policy violations and TN the number of compliant resources correctly accepted.

The False Negative Rate (FNR) measures the proportion of expected policy violations that remain undetected:

$$FNR = \frac{FN}{TP + FN} \times 100 \quad (3)$$

Policy Coverage (PC) measures the proportion of governance policies enforced by a given enforcement stage:

$$PC = \frac{P_{enforced}}{P_{total}} \times 100 \quad (4)$$

where $P_{enforced}$ denotes the number of governance policies enforced by the evaluated enforcement stage and P_{total} the total number of evaluated governance policies.

Finally, CI Bypass Protection (CBP) measures the effectiveness of Kubernetes admission control in mitigating CI bypass scenarios:

$$CBP = \frac{S_{blocked}}{S_{bypass}} \times 100 \quad (5)$$

where $S_{blocked}$ denotes the number of CI bypass scenarios correctly rejected by Kubernetes admission control and S_{bypass} the total number of evaluated CI bypass scenarios.

For all percentage-based metrics, values approaching 100% indicate more effective policy enforcement, whereas the ideal value for both the False Positive Rate and the False Negative Rate is 0%.

Figure 2 summarizes the complete experimental workflow adopted in this study. The workflow illustrates the relationship between the evaluation dataset, the shared policy-definition layer, the two complementary enforcement stages, and the quantitative evaluation process. By reusing identical policy definitions throughout all validation stages, the experimental design isolates the effects of policy reuse from those of the underlying enforcement technologies. Consequently, each research question is addressed through explicit, objective, and fully traceable experimental evidence.

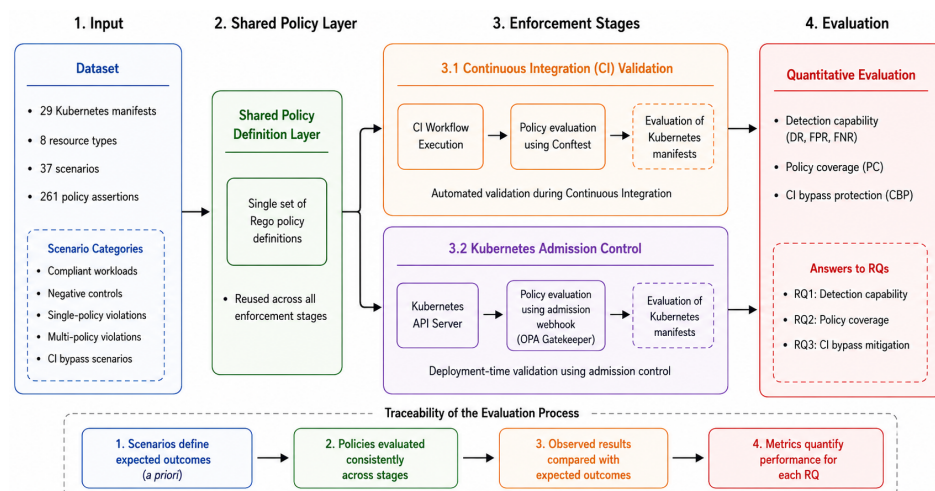


Figure 2. Overview of the experimental workflow adopted in this study. The evaluation uses a systematically constructed Kubernetes dataset together with a shared policy-definition layer implemented using Rego. The same policy definitions are reused during both Continuous Integration (CI) validation and Kubernetes admission control, ensuring that any observed differences arise from the execution context rather than from differences in policy implementation. Experimental results are quantified using well-defined evaluation metrics and mapped directly to the research questions through a fully traceable evaluation process.

3.6. Experimental Design

The experimental design was structured to evaluate the proposed reusable Policy-as-Code enforcement model under controlled and reproducible conditions while ensuring complete traceability between the research questions, the experimental scenarios, and the collected evidence. All experimental scenarios were executed using identical Kubernetes manifests, a common set of Rego policy definitions, and identical validation procedures across both enforcement stages. Consequently, any observed differences in policy decisions originate from the execution context rather than from differences in policy implementation.

Each experimental scenario followed a predefined execution protocol. Kubernetes manifests were first evaluated during Continuous Integration using ConfTest integrated into GitHub Actions workflows. The same manifests were subsequently submitted to a Kubernetes cluster protected by OPA Gatekeeper, where deployment-time policy decisions were enforced through Kubernetes validating admission webhooks. The observed policy decisions from both enforcement stages were then compared against the expected outcomes defined *a priori* for each scenario and quantified using the evaluation metrics introduced in the previous subsection.

Figure 2 provides an operational overview of the evaluation workflow, illustrating the relationship between the evaluation dataset, the shared policy-definition layer, the complementary enforcement stages, and the quantitative evaluation process. Together, the execution protocol and evaluation workflow establish complete traceability from the experimental artifacts to the empirical evidence used to answer each research question.

Table 4 summarizes the traceability between the research questions, the experimental scenarios, the quantitative evaluation metrics, and the empirical evidence collected throughout the evaluation. Detailed expected outcomes and their interpretation are presented in the corresponding evaluation subsections, ensuring that each research question is investigated through predefined experimental scenarios and objective evaluation criteria.

Table 4. Traceability between research questions, experimental scenarios, evaluation metrics, and collected evidence.

RQ	Scenarios	Metrics	Collected Evidence
RQ1	S01–S24	DR, FPR, FNR	Detection capability of the shared policy-definition layer
RQ2	S01–S29	PC	Policy coverage across complementary enforcement stages
RQ3	S25–S29	CBP	Admission-control effectiveness under CI bypass scenarios

3.7. Replication Package

To support reproducibility, transparency, and independent verification of the reported results, all artifacts developed for this study are publicly available through an open replication package. The repository contains the complete implementation of the proposed reusable multi-stage Policy-as-Code enforcement model together with the resources required to reproduce the experimental evaluation presented in this paper.

The replication package includes:

- The Rego policy definitions implementing the shared policy-definition layer;
- The Kubernetes manifests comprising the evaluation dataset;
- The experimental scenarios together with their expected policy outcomes;
- GitHub Actions workflows implementing Continuous Integration validation;
- OPA Gatekeeper constraints and Kubernetes admission-control configuration;
- The traceability matrices linking research questions, experimental scenarios, evaluation metrics, and collected evidence;
- Experimental results, execution instructions, and supporting documentation required to reproduce the evaluation.

Providing these artifacts publicly enables independent verification of the reported results while facilitating future extensions of the proposed evaluation methodology. Researchers and practitioners may reuse the evaluation dataset, policy definitions, validation workflows, and experimental methodology to investigate additional governance policies, Kubernetes resource types, complementary enforcement stages, or alternative Policy-as-Code ecosystems under comparable experimental conditions. The complete replication package is publicly available in a GitHub repository (<https://github.com/Impnogueira/policy-reuse-evaluation>, accessed on 1 July 2026).

Overall, the methodology presented in this section establishes a rigorous and reproducible experimental framework comprising a systematically constructed evaluation dataset, a reproducible execution environment, explicitly defined research questions, quantitative evaluation metrics, and complete traceability between the research questions, experimental scenarios, collected evidence, and reported results. This framework provides the methodological foundation for the empirical evaluation presented in the following sections and supports the systematic investigation of policy reuse across complementary enforcement stages in cloud-native software-delivery pipelines.

4. Reusable Multi-Stage Policy Enforcement Model

Building upon the research methodology presented in Section 3, this section introduces the reusable multi-stage Policy-as-Code enforcement model proposed in this work. The model addresses the research gap identified in Section 2 by investigating how reusable policy definitions can support consistent governance enforcement across complementary stages of the cloud-native software delivery lifecycle.

Rather than proposing a new Policy-as-Code technology, the model defines an architectural approach centered on a shared policy-definition layer reused across complementary enforcement stages. By separating policy definition from policy enforcement, governance requirements can evolve independently of the mechanisms responsible for their execution while preserving equivalent policy behavior throughout the software-delivery lifecycle.

The proposed architecture exploits the complementary characteristics of the two evaluated enforcement stages. Continuous Integration validation provides early detection of governance violations together with rapid developer feedback, whereas Kubernetes admission control performs authoritative deployment-time validation immediately before workload admission. Instead of treating these mechanisms as competing alternatives, the proposed model combines them through a common policy-definition layer that supports layered governance enforcement.

The remainder of this section presents the proposed architecture, describes its logical layers and complementary enforcement stages, and introduces the representative governance-policy portfolio used throughout the experimental evaluation.

4.1. Architectural Overview

Figure 3 presents the proposed reusable multi-stage Policy-as-Code enforcement model. The architecture is organized into three logical layers that separate policy definition from policy enforcement, enabling reusable policy definitions to be consistently evaluated across complementary enforcement stages throughout the cloud-native software delivery lifecycle. Table 5 summarizes the responsibilities of each architectural layer.

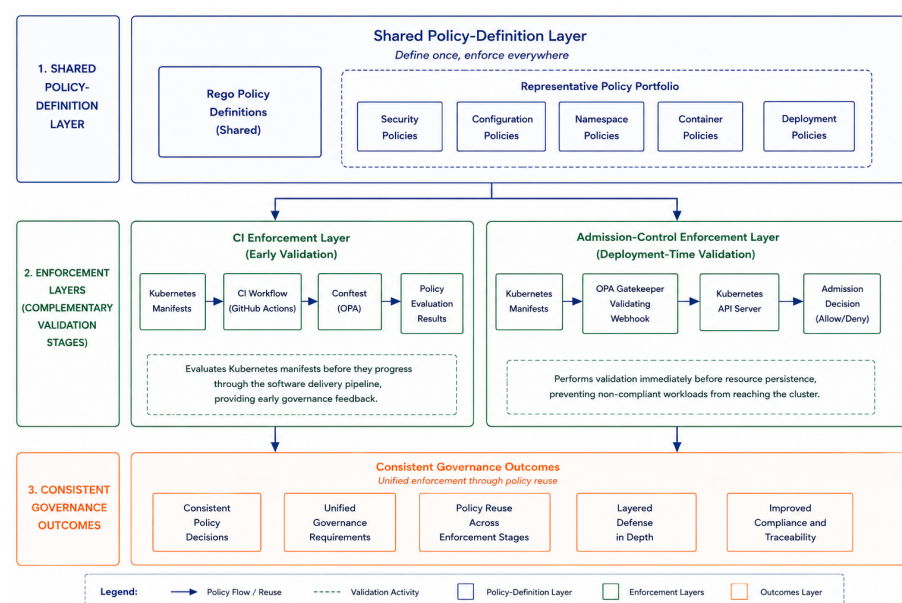


Figure 3. Reusable multi-stage Policy-as-Code enforcement model. A shared policy-definition layer is reused across complementary enforcement stages, enabling consistent governance validation during both Continuous Integration and Kubernetes admission control.

Table 5. Logical layers of the proposed enforcement model.

Layer	Responsibility
Shared Policy-Definition Layer	Defines reusable governance requirements independently of the enforcement mechanism through shared Rego policies.
Continuous Integration Enforcement Layer	Evaluates Kubernetes manifests before they progress through the software-delivery pipeline, providing early governance feedback.
Admission-Control Enforcement Layer	Performs deployment-time validation immediately before Kubernetes resource persistence, preventing non-compliant workloads from reaching the cluster.

The separation between policy definition and policy enforcement constitutes the fundamental architectural principle of the proposed model. Governance requirements are specified once within the shared policy-definition layer, which captures them independently of any specific enforcement technology, and are subsequently evaluated by multiple enforcement mechanisms without requiring duplicate policy implementations. As summarized in Table 5, the enforcement layers determine only when and where these requirements are evaluated, so the same governance logic can be reused across heterogeneous execution contexts and can evolve independently of the execution environment while preserving equivalent governance behavior across complementary enforcement stages.

Although the present study evaluates two complementary enforcement stages, the proposed architecture is intentionally extensible. Additional validation mechanisms may be incorporated into the software-delivery lifecycle, provided they support execution of the same policy definitions. Consequently, the architecture can accommodate additional enforcement technologies without requiring changes to the underlying governance requirements.

4.2. Governance Consistency Through Policy Reuse

The primary objective of the proposed model is to support governance consistency through policy reuse. In many cloud-native software-delivery environments, equivalent governance requirements are implemented independently within Continuous Integration pipelines, deployment platforms, infrastructure-validation tools, and runtime security mechanisms. Although these controls frequently pursue similar governance objectives, independently maintained policy implementations may gradually diverge as they evolve, leading to inconsistent policy decisions across different stages of the software delivery lifecycle.

The proposed architecture addresses this challenge through the same separation of policy definition from policy enforcement introduced above. Because governance requirements are specified once using Rego rather than reimplemented per enforcement mechanism, they can be enforced consistently throughout the software-delivery lifecycle without the divergence risk just described.

This extensibility, discussed above with respect to the overall architecture, applies equally to policy reuse across enforcement stages: additional validation technologies capable of evaluating the same policy definitions may be incorporated without requiring changes to the underlying governance requirements, allowing the architecture to evolve while preserving consistent governance behavior.

This architectural approach is consistent with recent Policy-as-Code and Security-as-Code research advocating governance artifacts that can be versioned, reviewed, tested, and maintained using established software-engineering practices [9–11]. The present work extends this perspective by investigating whether reusable policy definitions can support consistent governance enforcement across multiple complementary enforcement stages rather than within a single execution context.

By enabling governance requirements to be maintained independently of the mechanisms responsible for their enforcement, the proposed model establishes the architectural foundation for reducing duplicate policy implementations and limiting opportunities for policy drift. Although the present study does not directly quantify these potential organizational benefits, it provides empirical evidence regarding the practical feasibility of the proposed architectural approach under the evaluated experimental conditions.

4.3. Multi-Stage Enforcement Architecture

The proposed architecture operationalizes the shared policy-definition layer through two complementary enforcement stages positioned at different points of the cloud-native software-delivery lifecycle. Each stage fulfills a distinct operational role while evaluating the same governance requirements, thereby establishing multiple governance-validation boundaries throughout the software-delivery process.

The first enforcement stage performs pre-deployment validation during Continuous Integration (CI). Kubernetes manifests are evaluated before they progress through the software-delivery pipeline, enabling governance violations to be identified early in the development process. This stage provides rapid developer feedback, facilitating timely remediation before deployment activities commence.

The second enforcement stage performs deployment-time validation through Kubernetes admission control. Deployment requests are intercepted immediately before resource persistence within the Kubernetes API server, where governance policies are evaluated using validating admission webhooks [23]. This stage establishes an independent deployment-time enforcement boundary capable of preventing non-compliant workloads from reaching the target cluster, including situations in which upstream CI validation has been bypassed.

Table 6 summarizes the complementary responsibilities of the two enforcement stages. Continuous Integration validation focuses on preventing non-compliant Kubernetes manifests from progressing through the software-delivery pipeline, whereas Kubernetes admission control provides an independent deployment-time safeguard immediately before workload admission.

Table 6. Complementary roles of the two enforcement stages.

Enforcement Stage	Lifecycle Position	Primary Objective
Continuous Integration	Pre-deployment	Early detection of governance violations and rapid developer feedback.
Admission Control	Deployment-time	Prevent non-compliant workloads from being admitted to the Kubernetes cluster.

Together, these enforcement stages implement a layered defense-in-depth strategy in which identical governance requirements are evaluated at multiple points throughout the software-delivery lifecycle. Rather than replacing one another, they combine early developer feedback with authoritative deployment-time validation while relying on a common policy-definition layer to preserve equivalent governance behavior across both execution contexts.

The experimental evaluation presented in Section 5 investigates the practical feasibility of this architecture by assessing whether reusable policy definitions can be consistently enforced across both validation stages under controlled experimental conditions.

4.4. Policy Portfolio

To evaluate the proposed reusable multi-stage Policy-as-Code enforcement model, a representative portfolio of Kubernetes governance policies was implemented using a

shared policy-definition layer. The portfolio comprises five governance policies implemented through a total of nine Rego deny rules. Rather than providing exhaustive coverage of Kubernetes security practices, the selected policies represent common governance concerns frequently encountered in cloud-native software-delivery pipelines while exercising different categories of governance constraints.

Table 7 summarizes the policy portfolio used throughout the experimental evaluation, together with its enforcement across the two complementary validation stages. The selected policies were chosen because they represent governance controls commonly recommended in Kubernetes security guidance and widely adopted within cloud-native software-delivery pipelines. Collectively, they cover complementary aspects of workload governance, including container security, resource management, and deployment configuration, while remaining suitable for systematic evaluation under controlled experimental conditions.

Table 7. Policy portfolio and enforcement across complementary enforcement stages.

Policy	Name	Governance Objective	CI	Admission Control
P1	No Root Execution	Prevent execution of containers with root privileges.	✓	✓
P2	No Privilege Escalation	Prevent workloads from enabling privilege escalation.	✓	✓
P3	Resource Requests and Limits	Ensure explicit resource requests and limits for workload scheduling and runtime stability.	✓	–
P4	No Mutable Image Tags	Prevent deployment of workloads using mutable image tags such as <code>latest</code> .	✓	✓
P5	No Privileged Containers	Prevent execution of privileged containers with elevated host access.	✓	–

All policies are evaluated during Continuous Integration using the shared Rego policy definitions. During deployment-time validation, three representative policies are additionally enforced through OPA Gatekeeper constraints implementing the same governance requirements. This configuration was intentionally selected to evaluate policy reuse across complementary enforcement stages rather than to maximize admission-control policy coverage.

The remaining governance policies could also be implemented at the admission-control stage. Their omission from the evaluated prototype reflects the implementation scope adopted for this proof-of-concept rather than an inherent limitation of OPA Gatekeeper or the proposed architectural model. Consequently, the experimental evaluation focuses on the consistency of policy reuse across complementary enforcement stages rather than on the completeness of the implemented admission-control policy portfolio.

Accordingly, the objective of the selected policy portfolio is not to assess the completeness of Kubernetes governance controls or to compare alternative governance technologies or policy frameworks. Instead, it provides a representative, reproducible, and systematically constructed set of governance requirements that enables the empirical evaluation of policy reuse under controlled experimental conditions.

5. Experimental Evaluation

5.1. Evaluation Dataset

The proposed reusable multi-stage Policy-as-Code enforcement model was evaluated using the systematically constructed Kubernetes dataset introduced in Section 3. The dataset provides controlled and reproducible experimental conditions in which the expected outcome of every policy assertion is known *a priori*, enabling objective assessment of policy-detection capability, policy coverage across complementary enforcement stages, and the effectiveness of deployment-time admission control.

Table 8 summarizes the principal characteristics of the dataset. The evaluation comprises 29 Kubernetes manifests organized into 37 experimental scenarios spanning eight Kubernetes resource types. The policy portfolio introduced in Table 7 consists of five governance policies (P1–P5) implemented through nine Rego deny rules, resulting in a total of 261 policy assertions evaluated throughout the study.

Table 8. Characteristics of the evaluation dataset.

Characteristic	Value
Kubernetes manifests	29
Experimental scenarios	37
Kubernetes resource types	8
Governance policies (P1–P5)	5
Rego deny rules	9
Policy assertions	261

Figure 4 provides an overview of the dataset organization and illustrates how the experimental scenarios are distributed across the evaluation categories used to investigate the three research questions.

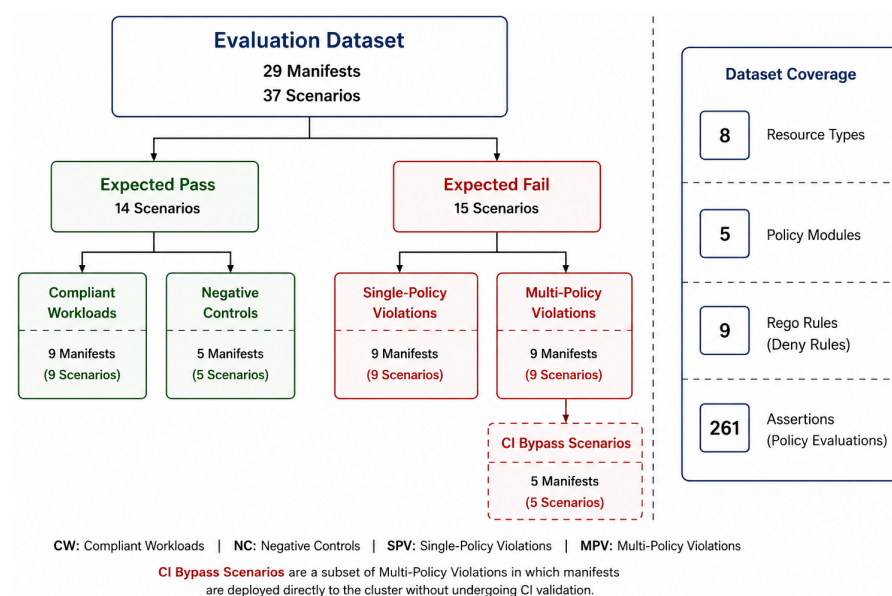


Figure 4. Organization of the evaluation dataset. The dataset comprises 29 Kubernetes manifests distributed across 37 experimental scenarios, including compliant workloads, negative controls, single-policy violations, multi-policy violations, and dedicated CI bypass scenarios. Collectively, these scenarios evaluate eight Kubernetes resource types, five governance policies (P1–P5) implemented through nine Rego deny rules, and a total of 261 policy assertions.

As illustrated in Figure 4, the experimental scenarios were intentionally organized to evaluate complementary aspects of the proposed enforcement model. Compliant workloads and negative controls assess policy-detection capability by verifying the absence of false-positive detections. Single-policy and multi-policy violation scenarios evaluate the ability of the shared policy definitions to identify governance violations under progressively more complex conditions. Finally, the dedicated CI bypass scenarios investigate the role of Kubernetes admission control as an independent deployment-time enforcement boundary capable of maintaining governance enforcement when upstream Continuous Integration validation is bypassed. Since these scenarios exercise the Kubernetes validating admis-

sion webhook directly, they specifically evaluate the resilience of the admission-control enforcement chain rather than all possible deployment bypass mechanisms.

Collectively, the dataset provides systematic coverage of the experimental conditions required to address the three research questions introduced in Section 3. The following subsections present the experimental evidence obtained for each research question, beginning with the evaluation of policy-detection capability during Continuous Integration validation (RQ1).

5.2. RQ1: Automatic Detection of Insecure Configurations

The first research question investigates whether the proposed reusable Policy-as-Code enforcement model can correctly identify the evaluated insecure Kubernetes configurations while avoiding false-positive detections on compliant workloads and unrelated resources. This evaluation focuses on the correctness of the shared policy definitions during Continuous Integration (CI) validation, independently of the deployment-time enforcement stage.

Figure 5 illustrates the CI validation workflow adopted throughout the experiments. Kubernetes manifests are evaluated against the shared Rego policy definitions during GitHub Actions execution using Conftest. Policy violations are reported before deployment artifacts progress through the software-delivery pipeline, enabling governance decisions to be enforced during pre-deployment validation.

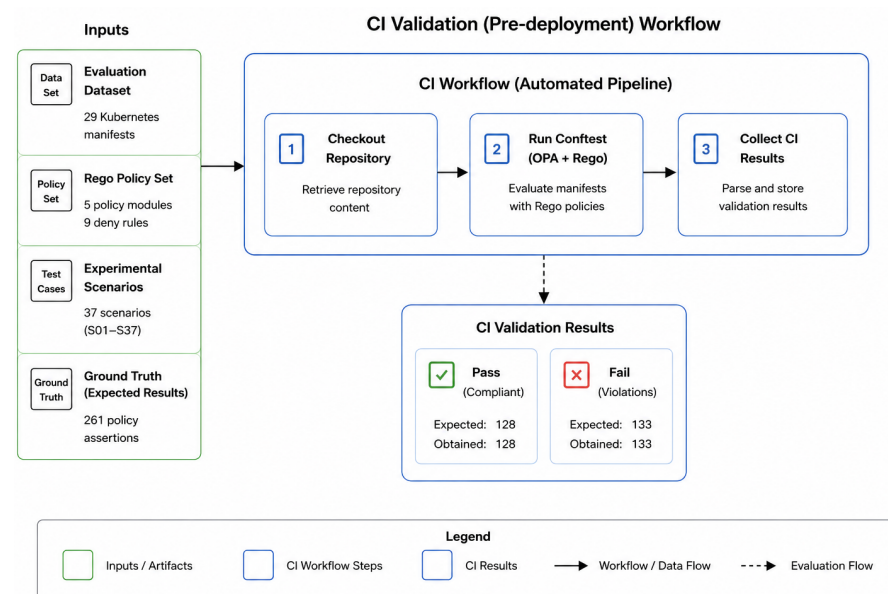


Figure 5. Continuous Integration Policy-as-Code validation workflow. Kubernetes manifests are evaluated against the shared policy-definition layer during GitHub Actions execution using Conftest. Policy violations prevent pipeline progression and provide immediate governance feedback before deployment.

The evaluation considered all 29 Kubernetes manifests comprising the experimental dataset. Fourteen manifests were expected to satisfy all evaluated governance policies (P1–P5), whereas 15 intentionally contained one or more governance violations. Together, these scenarios generated 40 expected policy violations, providing controlled experimental conditions for verifying the correctness of policy evaluation while quantifying false-positive and false-negative behavior.

Table 9 summarizes the quantitative results obtained for RQ1. Within the evaluated dataset, all expected policy violations were correctly identified without producing false-positive or false-negative detections. Consequently, the proposed approach achieved a

Detection Rate (DR) of 100%, a False Positive Rate (FPR) of 0%, and a False Negative Rate (FNR) of 0%.

Table 9. Detection results for RQ1.

Metric	Value
Policy portfolio evaluated	P1–P5
Kubernetes manifests evaluated	29
Expected policy violations	40
Detected policy violations	40
False positives	0
False negatives	0
Detection Rate (DR)	100%
False Positive Rate (FPR)	0%
False Negative Rate (FNR)	0%

To further assess the robustness of the implemented policy definitions, the evaluation included representative edge-case scenarios involving multi-container workloads, digest-based image references, omitted security fields, hardened security contexts, init containers, StatefulSets, and Jobs. All evaluated scenarios produced the expected policy decisions, indicating consistent governance behavior across different Kubernetes resource types and workload configurations within the scope of the evaluation dataset.

Overall, the experimental results provide empirical evidence that the proposed reusable Policy-as-Code enforcement model correctly identified all expected governance violations represented in the evaluation dataset while maintaining perfect detection accuracy under the evaluated experimental conditions. Although these findings should not be interpreted as characterizing detection performance across arbitrary Kubernetes workloads, they demonstrate the correctness of the implemented policy definitions and their ability to distinguish compliant from non-compliant manifests within the systematically constructed evaluation dataset. Consequently, RQ1 is answered affirmatively.

5.3. RQ2: Governance Assurance Through Multi-Stage Enforcement

The second research question investigates whether multi-stage Policy-as-Code enforcement provides broader governance assurance than relying on a single enforcement stage. Rather than comparing the individual effectiveness of Continuous Integration (CI) validation and Kubernetes admission control, the evaluation examines their complementary roles within the proposed architecture and the practical feasibility of consistently enforcing reusable Policy-as-Code definitions across multiple validation boundaries.

The experimental evaluation considered the five governance policies (P1–P5) introduced in Table 7. All policies were enforced during Continuous Integration validation using the shared Rego policy definitions, while three policies (P1, P2, and P4) were additionally implemented as OPA Gatekeeper constraints for deployment-time enforcement; the remaining two (P3 and P5) were evaluated only during CI, for the reasons discussed in Section 4.

Policy identifiers correspond to the governance portfolio defined in Table 7. As summarized in Table 10, Continuous Integration validation enforced the complete policy portfolio, corresponding to a Policy Coverage (PC) of 100%. Deployment-time admission control enforced three of the five governance policies implemented within the evaluated prototype, corresponding to a Policy Coverage of 60%. As discussed in Section 4, the remaining two policies remained exclusive to CI validation because their implementation as Gatekeeper constraints was outside the scope of the present proof-of-concept. Accordingly, the reported Policy Coverage values reflect the implementation scope adopted for the

evaluated prototype rather than inherent limitations of Kubernetes admission control, OPA Gatekeeper, or the proposed reusable Policy-as-Code enforcement model.

Table 10. Policy coverage across complementary enforcement stages.

Policy	CI	Admission	Scope
P1	✓	✓	Multi-stage
P2	✓	✓	Multi-stage
P3	✓	–	CI only
P4	✓	✓	Multi-stage
P5	✓	–	CI only
Policy Coverage (PC)	100%	60%	–

Although admission control enforced only a representative subset of the evaluated policy portfolio, the combination of both enforcement stages preserved enforcement of all governance policies considered in this study. More importantly, the same Rego policy definitions were reused across both execution contexts without requiring duplicate policy implementations. The remaining governance policies could likewise be implemented at the admission-control stage without requiring changes to the proposed architectural model, further confirming that the reported coverage differential reflects the scope of the evaluated prototype rather than the capabilities of the underlying enforcement mechanisms.

These results indicate that the principal contribution of the proposed architecture is not to increase the number of governance policies that can be enforced, but to demonstrate the practical feasibility of reusing common policy definitions across complementary enforcement stages. Continuous Integration validation provides comprehensive pre-deployment governance verification, whereas Kubernetes admission control reinforces deployment-time protection for representative governance requirements. Together, these complementary validation stages establish multiple governance-validation boundaries while preserving consistent governance behavior.

Consequently, RQ2 is answered affirmatively: within the evaluated implementation, reusable Policy-as-Code definitions were enforced consistently across both validation stages, with the reported Policy Coverage values reflecting the scope of the evaluated prototype rather than a capability limit of the underlying architecture.

5.4. RQ3: Admission Control and CI Bypass Mitigation

The third research question investigates whether Kubernetes admission control can maintain governance enforcement when deployment artifacts bypass Continuous Integration (CI) validation. This evaluation examines the role of admission control as an independent deployment-time enforcement boundary within the proposed reusable multi-stage Policy-as-Code architecture.

Figure 6 illustrates the admission-control workflow adopted throughout the experiments. Deployment requests submitted directly to the Kubernetes API server are intercepted by OPA Gatekeeper through Kubernetes validating admission webhooks and evaluated against the shared Rego policy definitions before resource persistence occurs.

To reproduce CI bypass conditions, deployment manifests were submitted directly to the Kubernetes API server using `kubectl apply-dry-run=server`. This approach exercises the Kubernetes validating admission webhook chain while intentionally bypassing the CI validation stage, allowing deployment-time policy enforcement to be evaluated independently. It therefore assesses the resilience of the admission-control enforcement chain rather than all possible deployment bypass mechanisms.

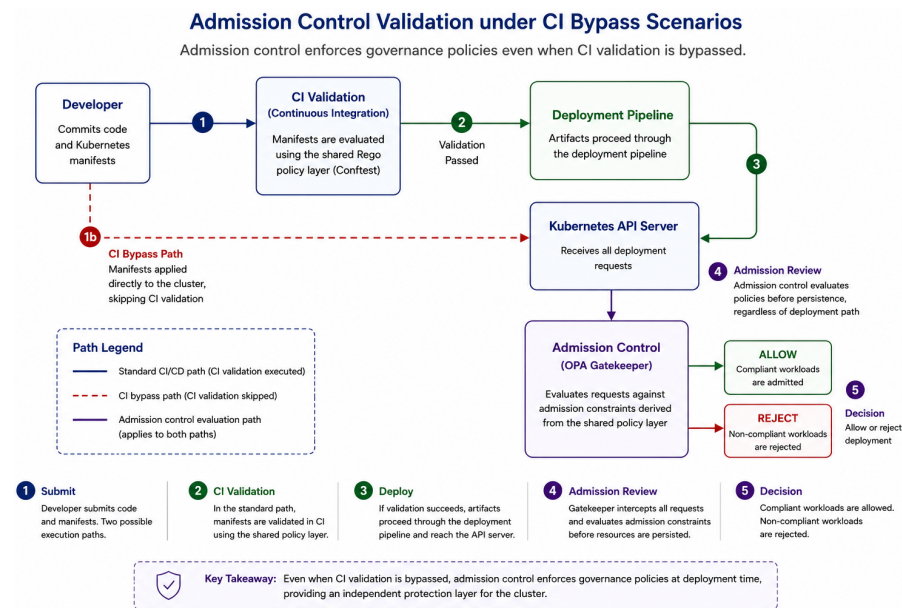


Figure 6. Admission-control validation workflow executed by OPA Gatekeeper. Deployment requests are intercepted immediately before resource persistence and evaluated against the shared policy-definition layer. Non-compliant workloads are rejected regardless of whether upstream CI validation has been executed.

Table 11 summarizes the results obtained for the CI bypass experiments. All five evaluated scenarios produced the expected outcome. The baseline compliant workload was successfully admitted, whereas every non-compliant workload was rejected before resource persistence. Consequently, the proposed approach achieved a CI Bypass Protection (CBP) of 100% for the evaluated scenarios.

Table 11. Admission-control results for CI bypass scenarios.

Scenario	Governance Condition	Expected	Observed
S25	Baseline compliant workload	Allowed	Allowed
S26	P1 – No Root Execution	Rejected	Rejected
S27	P2 – No Privilege Escalation	Rejected	Rejected
S28	P4 – No Mutable Image Tags	Rejected	Rejected
S29	P1 + P2 + P4 violations	Rejected	Rejected
CI Bypass Protection (CBP)		100%	

The multi-policy violation scenario provides additional evidence regarding the behavior of the admission-control mechanism. Rather than terminating evaluation after detecting the first violation, OPA Gatekeeper evaluated all applicable admission-control constraints associated with the deployment request, demonstrating consistent enforcement of the governance requirements implemented at the deployment stage.

These results demonstrate that Kubernetes admission control provides an effective independent deployment-time enforcement boundary capable of preserving governance enforcement even when upstream Continuous Integration validation is bypassed. Although the evaluated prototype implements deployment-time enforcement for only three representative governance policies (P1, P2, and P4), the experiments demonstrate the practical feasibility of the proposed defense-in-depth architecture under the evaluated experimental conditions.

Consequently, RQ3 is answered affirmatively: Kubernetes admission control complements Continuous Integration validation by providing an independent deployment-time

enforcement boundary, mitigating all evaluated CI bypass scenarios while preserving consistent governance enforcement for the supported deployment-time policy portfolio.

5.5. Summary of Findings

The experimental evaluation provides consistent empirical evidence supporting the three research questions investigated in this study. Across 29 Kubernetes manifests, 37 experimental scenarios, and 261 policy assertions, the proposed reusable Policy-as-Code enforcement model achieved a Detection Rate (DR) of 100% with a False Positive Rate (FPR) and False Negative Rate (FNR) of 0% (RQ1); Continuous Integration validation enforced the complete governance-policy portfolio (P1–P5) while Kubernetes admission control enforced the representative subset implemented within the evaluated prototype (P1, P2, and P4), with the two stages together preserving enforcement of the full portfolio through common policy definitions (RQ2); and Kubernetes admission control achieved a CI Bypass Protection (CBP) of 100% by correctly rejecting every non-compliant deployment submitted through the five evaluated CI bypass scenarios (RQ3).

Collectively, these results provide empirical evidence supporting the practical feasibility of policy reuse across complementary enforcement stages within a cloud-native software-delivery pipeline. Although the evaluation was intentionally limited to the selected governance policies, the implemented admission-control constraints, and the controlled experimental environment, the findings demonstrate that reusable Policy-as-Code definitions can preserve consistent governance behavior while supporting a layered defense-in-depth architecture spanning both pre-deployment and deployment-time validation. Section 6 discusses the broader implications of these findings, their limitations, and their relationship to existing Policy-as-Code research.

6. Discussion

6.1. Policy Reuse and Governance Consistency

The experimental results demonstrate that the principal contribution of the proposed approach lies not in introducing new Policy-as-Code technologies, but in showing that reusable policy definitions can be consistently enforced across complementary stages of the software-delivery lifecycle. Within the evaluated environment, identical governance requirements were enforced during both Continuous Integration (CI) validation and Kubernetes admission control, providing empirical evidence that policy definition can be effectively decoupled from policy enforcement.

This observation extends the current discussion surrounding Policy-as-Code. Existing research has primarily emphasized executable governance artifacts, compliance automation, and policy enforcement within individual execution contexts [9–11]. Similarly, studies addressing Kubernetes governance have generally evaluated admission control, static analysis, or CI validation as independent enforcement mechanisms [15,17]. Comparatively little empirical evidence has been available regarding whether common policy definitions can be reused consistently across multiple enforcement stages while preserving equivalent governance behavior.

The present study contributes to this discussion by demonstrating the practical feasibility of this architectural approach under controlled experimental conditions. Throughout the evaluation, the same Rego policy definitions were executed by different enforcement mechanisms without requiring duplicate policy implementations or policy-specific adaptations. Although the study does not quantify long-term organizational outcomes, it demonstrates that governance requirements can be maintained independently of the mechanisms responsible for their execution.

From a software-engineering perspective, this architectural separation offers several potential organizational advantages. Maintaining governance requirements within a common policy-definition layer may simplify policy evolution, reduce duplicated policy implementations, and limit opportunities for policy drift as governance requirements evolve over time. These potential benefits are frequently discussed in the Policy-as-Code literature [6,12,13], yet empirical evaluations of reusable policy-definition architectures have remained comparatively limited. The present study, therefore, provides initial empirical evidence supporting this architectural principle.

It is important to distinguish the scope of these findings. The reported results do not demonstrate that policy reuse automatically improves governance effectiveness, reduces operational costs, or simplifies long-term policy maintenance. Rather, they demonstrate that reusable Policy-as-Code definitions can preserve equivalent governance behavior across complementary enforcement stages under the evaluated experimental conditions. Quantifying the long-term impact of policy reuse on maintenance effort, policy evolution, governance consistency, and organizational processes remains an important direction for future research.

Overall, the findings reinforce the view that Policy-as-Code should be regarded not only as a mechanism for governance automation, but also as an architectural principle that separates governance definition from governance enforcement. This perspective broadens the role of Policy-as-Code beyond individual validation tools by positioning reusable policy definitions as shared governance artifacts capable of supporting layered governance throughout the cloud-native software-delivery lifecycle.

6.2. Implications of Multi-Stage Enforcement

The experimental results also speak to how the two enforcement stages contribute to governance assurance. Rather than increasing the number of governance policies that can be enforced, their principal value lies in extending policy enforcement across complementary stages of the software-delivery lifecycle. Instead of operating as competing alternatives, Continuous Integration (CI) validation and Kubernetes admission control fulfill distinct yet complementary governance functions that collectively strengthen the overall enforcement architecture.

The evaluation demonstrates that these two enforcement stages address different operational objectives. Continuous Integration validation supports early detection of governance violations and provides rapid feedback before deployment artifacts progress through the delivery pipeline, thereby reducing remediation effort and preventing non-compliant configurations from propagating downstream. Kubernetes admission control, by contrast, performs authoritative deployment-time validation immediately before resource persistence, ensuring that governance requirements remain enforceable even when upstream validation activities are bypassed.

These findings highlight an important architectural implication. The effectiveness of a multi-stage Policy-as-Code architecture should not be assessed solely in terms of policy-detection accuracy or the number of implemented governance policies. Instead, its principal value lies in establishing complementary validation boundaries that operate at different stages of the software-delivery lifecycle while evaluating common policy definitions. Under the evaluated experimental conditions, this layered approach increased resilience against governance failures by ensuring that deployment-time validation complemented, rather than duplicated, the protection provided by Continuous Integration validation.

These observations are consistent with the broader defense-in-depth principle widely adopted in cybersecurity, whereby multiple independent protection mechanisms collectively improve system resilience. Within the proposed architecture, the complementary

enforcement stages reduce reliance on any single validation mechanism while preserving a common set of policy definitions. Consequently, equivalent governance behavior can be maintained even when one validation boundary is intentionally or unintentionally bypassed.

More broadly, the findings suggest that reusable Policy-as-Code architectures may provide organizational benefits beyond governance automation alone. By separating policy definition from policy enforcement, organizations may evolve governance requirements independently of individual validation mechanisms while preserving consistent governance behavior across heterogeneous software-delivery environments. Although the present study does not directly evaluate long-term organizational outcomes such as reduced maintenance effort, simplified policy evolution, or lower operational costs, it demonstrates the practical feasibility of this architectural approach under the evaluated implementation scope and provides a foundation for future empirical investigations involving larger policy portfolios, production-scale environments, and alternative Policy-as-Code ecosystems.

6.3. Admission Control as a Defense-in-Depth Mechanism

The CI bypass experiments provide some of the strongest empirical evidence presented in this study because they evaluate the proposed architecture under conditions in which the primary pre-deployment validation stage is intentionally circumvented. Rather than assessing policy evaluation alone, these experiments investigate whether layered governance can be maintained when one of the validation boundaries is absent.

Traditional DevSecOps workflows frequently rely on Continuous Integration (CI) validation as the primary governance-enforcement mechanism. In practice, however, deployment artifacts may reach downstream environments through alternative deployment paths, operational workarounds, emergency procedures, configuration errors, or temporary process exceptions. Under such conditions, relying exclusively on CI validation may leave production environments exposed to non-compliant deployments.

The experimental results demonstrate that Kubernetes admission control provides an effective independent deployment-time enforcement boundary. In every evaluated CI bypass scenario, non-compliant workloads were rejected before resource persistence within the Kubernetes cluster, thereby preserving governance enforcement despite the absence of upstream CI validation. These findings indicate that deployment-time admission control complements, rather than replaces, earlier validation activities by providing an additional validation boundary immediately before workload admission.

The evaluated CI bypass scenarios specifically exercise the Kubernetes validating admission webhook chain through server-side validation using `kubectl apply-dry-run=server`. Consequently, the reported results demonstrate the effectiveness of deployment-time admission control within the evaluated admission workflow and should not be interpreted as covering every possible deployment bypass mechanism or unauthorized deployment path.

From an architectural perspective, these observations reinforce the applicability of the defense-in-depth principle to Policy-as-Code governance in cloud-native software-delivery environments [24]. Rather than relying on a single validation stage, governance requirements are evaluated across multiple complementary validation boundaries while reusing the same policy definitions. This layered architecture reduces dependence on any individual control mechanism and preserves equivalent governance behavior even when one validation boundary is intentionally bypassed.

It is important, however, to interpret these findings within the scope of the evaluated implementation. The admission-control experiments considered only the representative subset of governance policies implemented as OPA Gatekeeper constraints together with a controlled set of CI bypass scenarios. Consequently, although the results demonstrate the practical feasibility of this defense-in-depth approach under the evaluated experimen-

tal conditions, further studies involving larger policy portfolios, additional deployment paths, and production-scale infrastructures are required to assess the generalizability of these observations.

6.4. Threats to Validity and Limitations

The findings of this study should be interpreted in light of several threats to validity and limitations associated with the experimental design, the evaluated policy portfolio, and the implementation scope.

Regarding internal validity, the experimental evaluation employed deterministic Policy-as-Code definitions together with predefined experimental scenarios for which the expected outcome of every policy assertion was known *a priori*. To minimize execution-related inconsistencies, all experiments reused the same Rego policy definitions, identical validation procedures, and equivalent deployment artifacts across both Continuous Integration (CI) validation and Kubernetes admission control.

The evaluation dataset and the corresponding expected policy outcomes were constructed by the same authors who implemented the evaluated Policy-as-Code definitions. Although every experimental scenario specifies its expected outcome *a priori* and the complete replication package enables independent verification of the reported results, the absence of an independent validation of the expected outcomes represents a potential source of experimenter bias.

Regarding construct validity, the selected evaluation metrics were designed to provide objective evidence for answering the three research questions. This was supported by the explicit traceability established in Section 3 between the research questions, experimental scenarios, evaluation metrics, and expected outcomes. Nevertheless, governance effectiveness was assessed primarily through policy evaluation, policy coverage, and protection against CI bypass scenarios. Other potentially relevant dimensions, including policy maintainability, policy evolution, organizational adoption, developer productivity, and operational cost, were beyond the scope of the present study.

Regarding external validity, the evaluation employed a systematically constructed dataset comprising 29 Kubernetes manifests and 37 experimental scenarios. Although the dataset intentionally covers compliant workloads, negative controls, single-policy violations, multi-policy violations, and CI bypass scenarios, it represents a controlled experimental environment rather than a statistically representative production deployment. Accordingly, the reported findings should not be interpreted as characterizing governance behavior across arbitrary Kubernetes environments.

Furthermore, the evaluated scenarios were intentionally designed by the authors rather than derived from operational incident datasets or production repositories. Although this design improves experimental control and reproducibility, it may not capture the full diversity of governance violations encountered in operational cloud-native environments.

A further limitation concerns the evaluated policy portfolio. The study considered five representative Kubernetes governance policies implemented through nine Rego `deny` rules. Although these policies address common workload-security and configuration-governance concerns, they do not encompass the broader range of organizational, regulatory, or domain-specific governance requirements encountered in large-scale cloud-native environments.

The implementation was intentionally restricted to a single Policy-as-Code ecosystem based on Open Policy Agent (OPA), Rego, Conftest, and OPA Gatekeeper. Although the proposed architectural model is conceptually independent of these technologies, the reported evidence is limited to the evaluated ecosystem. Additional empirical studies are therefore required to assess the applicability of the proposed architectural principles across

alternative Policy-as-Code languages, policy engines, admission-control frameworks, and software-delivery platforms.

Finally, the evaluation focused on functional correctness and governance consistency rather than operational performance. Consequently, the impact of reusable Policy-as-Code definitions on admission-control latency, policy-evaluation overhead, resource consumption, scalability, and overall system performance was not investigated. These aspects require dedicated empirical evaluation under production-scale workloads.

Despite these limitations, several characteristics strengthen the credibility of the reported findings. The study employed a systematically constructed evaluation dataset, explicitly defined research questions, quantitative evaluation metrics, complete traceability between experimental artifacts and reported results, and a publicly available replication package containing the policy definitions, Kubernetes manifests, workflows, and supporting documentation. Collectively, these elements provide a transparent and reproducible foundation for assessing the proposed reusable multi-stage Policy-as-Code enforcement architecture.

6.5. Future Research Directions

The findings of this study suggest several promising directions for extending both the proposed architectural approach and the empirical evidence supporting reusable Policy-as-Code strategies.

A natural continuation of this work involves evaluating the proposed architecture using substantially larger datasets derived from public Kubernetes repositories, industrial deployments, and production-oriented software-delivery pipelines. Such studies would strengthen the external validity of the reported findings while providing additional evidence regarding the scalability, robustness, and practical applicability of policy reuse under realistic deployment conditions.

Future evaluations should also investigate the proposed architecture in large-scale industrial environments comprising hundreds of microservices, multiple development teams, and heterogeneous deployment pipelines. Evaluating the approach under these conditions would provide valuable evidence regarding its long-term applicability and governance consistency in production cloud-native systems.

Another promising direction is to investigate policy reuse across governance domains beyond Kubernetes workload validation. Candidate application areas include Infrastructure-as-Code verification, GitOps deployment workflows, software supply-chain governance, identity and access management, and multi-cloud policy enforcement. Extending the proposed architectural principles across these complementary domains would provide valuable insight into the broader applicability of reusable Policy-as-Code strategies throughout the software-delivery lifecycle.

Interoperability across heterogeneous Policy-as-Code ecosystems also warrants further investigation. Although the present study demonstrates successful policy reuse within the Open Policy Agent ecosystem, future work should examine whether equivalent governance behavior can be achieved across alternative policy languages, policy engines, admission-control frameworks, and software-delivery platforms. Comparative evaluations involving platforms such as Kyverno would be particularly valuable for assessing the generalizability of the proposed architectural principles beyond a single technological ecosystem.

Operational performance represents another important avenue for future work. The present study focused on functional correctness and governance consistency rather than runtime characteristics. Future investigations should therefore evaluate the impact of reusable Policy-as-Code definitions on policy-evaluation latency, admission-control throughput, resource consumption, and scalability under production-scale workloads. Particular attention

should be devoted to the latency introduced by validating admission webhooks under high request concurrency, since excessive webhook latency may affect Kubernetes control-plane responsiveness in large-scale production clusters. Such studies would complement the functional evaluation presented in this paper by characterizing the operational trade-offs associated with reusable multi-stage policy enforcement.

More broadly, the proposed architecture provides a foundation for investigating Policy-as-Code governance across multiple platforms, technologies, and enforcement stages. As cloud-native software-delivery ecosystems continue to evolve, understanding how reusable policy definitions can support consistent governance across increasingly heterogeneous operational environments represents a promising direction for future research.

7. Conclusions

This paper investigated whether reusable Policy-as-Code definitions can support consistent governance across complementary stages of the cloud-native software-delivery lifecycle. Although Policy-as-Code has become a widely adopted approach for automating security, compliance, and governance, comparatively little empirical evidence has been available regarding the reuse of common policy definitions across multiple enforcement mechanisms.

To address this research gap, this work proposed and empirically evaluated a reusable multi-stage Policy-as-Code enforcement architecture centered on a common policy-definition layer executed during both Continuous Integration validation and Kubernetes admission control. The proposed architecture was implemented using Open Policy Agent, Rego, Conftest, and OPA Gatekeeper and evaluated through a structured experimental study comprising 29 Kubernetes manifests, 37 experimental scenarios, eight Kubernetes resource types, and 261 policy assertions.

The experimental evaluation provided positive evidence for all three research questions, as detailed in Section 5: perfect detection accuracy on the evaluated dataset with no observed false positives or false negatives (RQ1), consistent enforcement of the policy portfolio across both stages through a single set of reused policy definitions (RQ2), and full mitigation of the evaluated CI bypass scenarios by deployment-time admission control (RQ3).

Beyond these individual findings, the principal contribution of this work lies in demonstrating the practical feasibility of separating policy definition from policy enforcement: common policy definitions can be reused across complementary enforcement mechanisms while preserving equivalent governance behavior, providing a reusable foundation for layered, defense-in-depth Policy-as-Code governance without requiring duplicate policy implementations.

The reported findings should be interpreted within the scope of the evaluated environment, which consisted of a controlled experimental dataset and a single Policy-as-Code ecosystem based on Open Policy Agent, Rego, Conftest, and OPA Gatekeeper. Accordingly, additional empirical studies involving larger datasets, production-scale deployments, alternative Policy-as-Code ecosystems, and heterogeneous software-delivery platforms are required to assess the broader applicability and generalizability of the proposed architectural approach. More broadly, these findings establish a reproducible foundation for future research on scalable, technology-independent, and reusable governance architectures spanning heterogeneous cloud-native software-delivery environments.

Author Contributions: Conceptualization, L.N.; Methodology, L.N. and A.R.; Software, A.R.; Validation, A.R.; Formal Analysis, L.N.; Investigation, A.R.; Data Curation, A.R.; Writing—Original Draft Preparation, L.N.; Writing—Review and Editing, L.N. and A.R.; Supervision, L.N. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The policies, Kubernetes manifests, experimental scenarios, CI workflows, admission-control configurations, validation results, and supporting artifacts used in this study are publicly available at: <https://github.com/Impnogueira/policy-reuse-evaluation>, accessed on 1 July 2026.

Acknowledgments: ChatGPT (4.0) (OpenAI) was used during the preparation of the manuscript to assist with language editing, text refinement, grammar correction, improvements in clarity and readability, and the refinement of figures and supporting documentation. AI tools were not used to generate experimental results, perform data analysis, design the experimental evaluation, implement the evaluated artifacts, interpret findings, or draw scientific conclusions. All technical content, experimental design, implementation, analysis, and final editorial decisions were carried out and verified by the authors.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Sánchez-Gordón, M.; Colomo-Palacios, R. Security as Culture: A Systematic Literature Review of DevSecOps. In Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops (ICSEW), Seoul, Republic of Korea, 27 June–19 July 2020; pp. 266–269. [\[CrossRef\]](#)
2. Akbar, M.A.; Smolander, K.; Mahmood, S.; Alsanad, A. Toward Successful DevSecOps in Software Development Organizations: A Decision-Making Framework. *Inf. Softw. Technol.* **2022**, *147*, 106894. [\[CrossRef\]](#)
3. Alonso, J.; Piliszek, R.; Cankar, M. Embracing IaC Through the DevSecOps Philosophy: Concepts, Challenges, and a Reference Framework. *IEEE Softw.* **2023**, *40*, 56–62. [\[CrossRef\]](#)
4. Billawa, P.; Bambhore Tukaram, A.; Diaz Ferreyra, N.E.; Steghöfer, J.P.; Scandariato, R.; Simhandl, G. SoK: Security of Microservice Applications: A Practitioners' Perspective on Challenges and Best Practices. In Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES 2022), Vienna, Austria, 23–26 August 2022; pp. 1–10.
5. Moyón, F.; Angermeir, F.; Mendez, D. Industrial Challenges in Secure Continuous Development. In Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP'24), Lisbon, Portugal, 14–20 April 2024; pp. 309–311. [\[CrossRef\]](#)
6. Ramaj, X.; Sánchez-Gordón, M.; Gkioulos, V.; Colomo-Palacios, R. On DevSecOps and Risk Management in Critical Infrastructures: Practitioners' Insights on Needs and Goals. In Proceedings of the International Workshop on Engineering and Cybersecurity of Critical Systems and the International Workshop on Software Vulnerability (EnCyCriS/SVM'24), Lisbon, Portugal, 15 April 2024; pp. 45–52. [\[CrossRef\]](#)
7. Prates, L.; Pereira, R. DevSecOps practices and tools. *Int. J. Inf. Secur.* **2025**, *24*, 11. [\[CrossRef\]](#)
8. Akbar, M.A.; AlSanad, A.A. Empirical Investigation of Key Enablers for Secure DevOps Practices. *IEEE Access* **2025**, *13*, 43698–43715. [\[CrossRef\]](#)
9. Haverinen, H.; Janhunen, T.; Päiväranta, T.; Lempinen, S.; Kaartinen, S.; Merilä, S. Automating Cybersecurity Compliance in DevSecOps with Open Information Model for Security as Code. In Proceedings of the 4th Eclipse Security, AI, Architecture and Modelling Conference on Data Spaces (eSAAM '24), Mainz, Germany, 22 October 2024; pp. 93–102. [\[CrossRef\]](#)
10. Vakhula, O.; Opirskyy, I.; Vorobets, P.; Bobko, O.M.; Kulinich, O. Research on Policy-as-Code for Implementation of Role-based and Attribute-based Access Control. *Cybersecur. Provid. Inf. Telecommun. Syst.* **2025**, *3991*, 139–157.
11. Vijayaraghavan, S.K.J. Policy as Code: A Paradigm Shift in Infrastructure Security and Governance. In *Proceedings of the ICT for Global Innovations and Solutions*; Bhattacharya, S., Ed.; Springer: Cham, Switzerland, 2025; pp. 182–193.
12. Gama, D.; Brito, A.; Martin, A.; Fetzter, C. Supporting continuous vulnerability compliance through automated identity provisioning. In Proceedings of the 13th Latin-American Symposium on Dependable and Secure Computing (LADC'24), Recife, Brazil, 26–29 November 2024; pp. 101–110. [\[CrossRef\]](#)
13. Port, D.; Taber, B.; Emkani, P. Investigating effectiveness and compliance to DevOps policies and practices for managing productivity and quality variability. *J. Syst. Softw.* **2024**, *213*, 112030. [\[CrossRef\]](#)
14. Open Policy Agent Contributors. Open Policy Agent Documentation. Available online: <https://www.openpolicyagent.org/docs/> (accessed on 15 July 2026).
15. Kapetanidou, I.A.; Nizamis, A.; Votis, K. An Evaluation of Commonly Used Kubernetes Security Scanning Tools. In Proceedings of the 2nd International Workshop on MetaOS for the Cloud-Edge-IoT Continuum (MECC '25), Rotterdam, The Netherlands, 30 March–3 April 2025; pp. 20–25. [\[CrossRef\]](#)

16. Jansen, M.; Talluri, S.; Doekemeijer, K.; Tehrany, N.; Iosup, A.; Trivedi, A. Columbo: A Reasoning Framework for Kubernetes' Configuration Space. In Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE '25), Toronto, ON, Canada, 5–9 May 2025; pp. 45–57. [[CrossRef](#)]
17. Rossi, M.; Beretta, M.; Facchinetti, D.; Paraboschi, S. POSTER: Policy-driven security-aware scheduling in Kubernetes. In Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (ASIA CCS '25), Hanoi, Vietnam, 25–29 August 2025; pp. 1800–1802. [[CrossRef](#)]
18. Open Policy Agent Contributors. Gatekeeper Documentation. Available online: <https://open-policy-agent.github.io/gatekeeper/website/> (accessed on 15 July 2026).
19. Ramaj, X. A DevSecOps-Enabled Framework for Risk Management of Critical Infrastructures. In Proceedings of the IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE Companion'22), Pittsburgh, PA, USA, 21–29 May 2022; pp. 242–244. [[CrossRef](#)]
20. The Kubernetes Authors. Admission Controllers. 2025. Available online: <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/> (accessed on 15 June 2026).
21. Hevner, A.R.; March, S.T.; Park, J.; Ram, S. Design Science in Information Systems Research. *MIS Q.* **2004**, *28*, 75–105. [[CrossRef](#)]
22. Peffers, K.; Tuunanen, T.; Rothenberger, M.A.; Chatterjee, S. A Design Science Research Methodology for Information Systems Research. *J. Manag. Inf. Syst.* **2007**, *24*, 45–77. [[CrossRef](#)]
23. The Kubernetes Authors. Dynamic Admission Control. 2025. Available online: <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/> (accessed on 15 June 2026).
24. Anderson, R. *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed.; Wiley: Indianapolis, IN, USA, 2020.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.